Supplementary Material: Detailed Heuristic, BAN Logic, ProVerif, and Provable-Security Analysis for Blockchain-based Cross-layer Authentication Scheme for 5G Network

Chenyu Wang, Dinghao Wang, and Shaoyong Guo

Beijing University of Posts and Telecommunications, Beijing 100876, China
wangchenyu@bupt.edu.cn

This supplementary document provides the detailed heuristic analysis, ProVerif model, BAN-logic derivations, and ROR-based provable-security analysis omitted from the conference paper because of space limitations. The presentation follows the same WASA/LNCS formatting style as the main manuscript so that the supplementary analysis can be read as a natural extension of the paper.

1 Detailed Heuristic Analysis

This section evaluates the proposed scheme against the nine functional and security criteria defined in the main paper. The reasoning is based on the scheme mechanisms of authentication-key updating, ECIES-based identity protection, blockchain-based public-key verification, and $Interp(\cdot)$-based physical-layer binding.

1.1 Key Agreement

After the challenge-response procedure is completed, the UE and HN derive k_{seaf} from the same authentication key, random challenge, sequence number, and serving-network name. The HN transmits the corresponding key material to the SN over the protected SN–HN channel, and the UE and SN subsequently perform key confirmation. Therefore, a successful protocol run results in a consistent session-key context for subsequent communication.

1.2 Mutual Authentication

The UE authenticates the HN by checking the sequence-number freshness and by comparing the received mac with the locally generated value

$$xmac = Interp(XMAC, H_{01}(f)), \qquad diff(mac, xmac) \leq \Delta. \tag{1}$$

The SN verifies $hres^*$ against $Interp(HXRES^*, H_{11}(f))$, and the HN verifies res^* against $Interp(XRES^*, H_{11}(f))$. Consequently, successful authentication requires both valid upper-layer AKA parameters and physical-layer evidence consistent with the current wireless channel.

1.3 System Friendliness

The scheme preserves the main 5G-AKA challenge-response framework and retains core parameters such as $RAND$, $AUTN$, RES^*, $HXRES^*$, and k_{seaf}. Blockchain retrieval, ECIES protection, authentication-key updating, and channel-binding operations are added around the existing procedure rather than replacing the overall architecture. Therefore, the scheme can be integrated without major changes to the 5G authentication workflow.

1.4 User Anonymity

The permanent identity $SUPI$ is protected by ECIES and is transmitted over the access link only as $SUCI$. An adversary that observes public authentication messages cannot recover the plaintext identity without the HN private key. Moreover, the random challenge, dynamically derived authentication key, and channel-bound responses generated by $Interp(\cdot)$ vary across sessions, preventing the adversary from extracting a stable identifier from repeated protocol executions.

1.5 Forward Secrecy

The authentication key is periodically derived as $k = Hash(S \parallel RAND_{i-1})$, where S is the ECDH shared secret computed from the UE/HN key pairs. Compromise of a later authentication key does not directly reveal S or an earlier freshness input. Recovering the ECDH secret from the public keys requires solving the elliptic-curve discrete logarithm problem, while earlier session-specific inputs are not recoverable from the current key alone. Hence, exposure of a later key does not enable efficient reconstruction of previous authentication keys or established session keys.

1.6 Resistance to Linkability Attacks

The challenge and response values, including $RAND'$, $AUTN'$, mac, res^*, and $hres^*$, are refreshed by the random challenge and the current channel observations. A replayed or out-of-session challenge fails at least one of the MAC check, sequence-number freshness check, or interpolation-based threshold test. The UE therefore does not produce a valid response that can be used to link different sessions.

1.7 Resistance to Impersonation Attacks

The public keys of the UE and HN are maintained as trusted mappings on the blockchain, preventing an attacker from substituting a forged public key for a legitimate identity. In addition, a successful impersonator would need to generate authentication values consistent with both the protected upper-layer parameters and the current UE–SN channel feature. Without the legitimate private key and the corresponding channel observation, the attacker cannot construct a response that passes all checks.

1.8 Resistance to Man-in-the-Middle Attacks

The UE and HN derive authentication material from verified public keys, private keys, and fresh challenges. A man-in-the-middle adversary cannot replace a public key without violating the blockchain mapping, nor can it compute the correct ECDH shared secret without a legitimate private key. Any modification of $RAND'$, $AUTN'$, res^*, or $hres^*$ breaks either the upper-layer AKA verification or the interpolation-based physical-layer matching.

1.9 Resistance to Collusive Attacks

Even if multiple adversaries share intercepted messages or partial session information, they cannot recover the UE/HN private keys, the ECDH shared secret, or the legitimate channel feature observed by the communicating entities. The blockchain prevents colluding attackers from forging a valid identity-to-key mapping, while fresh randomness and channel-bound responses prevent the reuse of collected data to construct valid authentication messages.

2 Detailed ProVerif Verification

To present the symbolic verification first, this section defines the communication channels and cryptographic primitives, then encodes the UE, HN, and SN behaviors, and finally issues secrecy and correspondence queries. This organization makes it possible to check both confidentiality and authentication in the standard symbolic-adversary model. The modeling strategy follows the standard Dolev–Yao abstraction: cryptographic primitives are treated as ideal symbolic operators, while the adversary is allowed to intercept, replay, and synthesize messages on public channels. This abstraction is well suited to the present protocol because the main goal is to confirm that the message flow, key dependencies, and event correspondences are structurally sufficient to preserve secrecy and mutual authentication.

In the ProVerif model, the UE–SN link is intentionally left public to reflect the exposure of the wireless side, whereas the SN–HN link is modeled as private in accordance with the trust assumptions adopted in the main paper. The code fragments reproduced below are not intended to exhaust every implementation detail; instead, they retain the essential computations that determine whether the authentication run can be completed and whether the derived session key material remains protected.

2.1 Premises and Security Queries

The main ProVerif premises used in the model are shown below.

Listing 1.1: Core ProVerif premises and secrecy queries.

```
free UE_SN: channel.
free SN_HN: channel [private].

type skey.
fun pk(skey): pkey.
fun aenc(bitstring, pkey): bitstring.
reduc forall m: bitstring, sk: skey;
      adec(aenc(m, pk(sk)), sk) = m.

fun challenge(key, nonce, SNID): xres_star.
fun keyseed(key, nonce, SQN, SNID): key_seaf.
fun f1(key, bitstring): MAC.
fun f5(key, nonce): AK.

query attacker(k_1);
      attacker(k_2);
      attacker(sk_hn);
      attacker(supi_1);
      attacker(supi_2).
query secret sqn_ue;
      secret sqn_hn;
      secret k_seaf.
```

These declarations capture the insecure UE–SN channel, the private SN–HN channel, the asymmetric encryption/decryption used by ECIES, the challenge/key-derivation functions, and the secrecy goals on $SUPI$, SQN, and k_{seaf}. In particular, the secrecy queries are aligned with the main security claims of the paper: they test whether the symbolic attacker can derive the long-term key material, user identity, sequence numbers, or the final access-session key. The model therefore checks not only confidentiality of static data but also the confidentiality of the transient values that drive successful 5G authentication.

2.2 Modeled Principal Processes

The three principal processes below mirror the functional split used in the main scheme description. The UE protects the subscriber identity, verifies the challenge, and generates the response; the HN rebuilds the subscriber state and prepares the challenge vector; and the SN mainly forwards messages while recording the authentication events used later in the correspondence queries. This organization keeps the symbolic model close to the real protocol logic and makes the role of each principal easier to follow.

Listing 1.2: UE process used in the ProVerif model.

```
let processUE(k: key, supi_ue: SUPI, pkHN: pkey, idHN: hostid) =
  new sqn: SQN;
  new Rs: nonce;
  in(UE_SN, SNname: SNID);
  let suci_ue = encapsulate_to_suci(aenc((supi_ue, Rs, sqn), pkHN), idHN) in
  out(UE_SN, suci_ue);
  in(UE_SN, m4: bitstring);
  let (rand: nonce, autn: AUTN) = m4 in
  let encapsulate_to_autn(xconc, xmac) = autn in
  let ak: AK = f5(k, rand) in
  let xsqn: SQN = dxor(xconc, ak) in
  let xmac: MAC = f1(k, (xsqn, rand, SNname)) in
  if mac = xmac then
    if sqn = xsqn then
      let res_star: xres_star = challenge(k, rand, SNname) in
      let k_seaf: key_seaf = keyseed(k, rand, xsqn, SNname) in
      out(UE_SN, res_star).
```

The UE process above highlights the two essential verification points on the user side: the symbolic counterpart of MAC checking and the confirmation that the sequence-number relation is valid. Only when both conditions hold does the UE produce res^* and the session key seed, which is consistent with the challenge–response logic of the scheme.

Listing 1.3: HN process used in the ProVerif model.

```
let processHN(skHN: skey, idHN: hostid) =
  in(SN_HN, m2: bitstring);
  let (suci: SUCI, SNname: SNID) = m2 in
  if id = idHN then
    let (supi_hn: SUPI, Rs: nonce, sqn: SQN) = adec(x, skHN) in
    get hss(=supi_hn, k_hn: key) in
    new rand: nonce;
    let mac: MAC = f1(k_hn, (sqn, rand, SNname)) in
    let ak: AK = f5(k_hn, rand) in
    let xres: xres_star = challenge(k_hn, rand, SNname) in
    let hxres: bitstring = sha256((rand, xres)) in
    let k_seaf: key_seaf = keyseed(k_hn, rand, sqn, SNname) in
    out(SN_HN, (rand, autn, hxres, k_seaf));
    in(SN_HN, m6: bitstring);
    if res = xres then out(SN_HN, supi_hn).
```

At the HN side, the model reconstructs the concealed subscription information, generates fresh challenge material, and derives the values that later drive authentication on both the UE and SN sides. This symbolic process is important because it binds the user identity, the challenge, and k_{seaf} to the same protocol run.

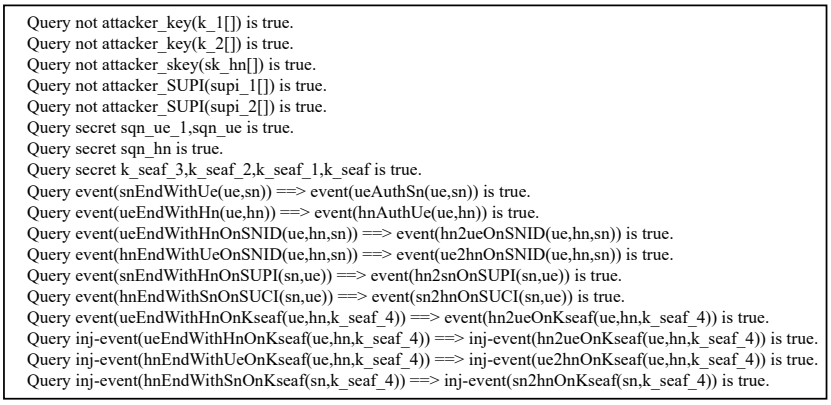

Fig. 1: Representative ProVerif verification result reproduced in the supplementary material.

Listing 1.4: SN process and representative authentication events.

```
let processSN(SNname: SNID) =
  out(UE_SN, SNname);
  in(UE_SN, suci: SUCI);
  out(SN_HN, (suci, SNname));
  in(SN_HN, m3: bitstring);
  let (rand: nonce, autn: AUTN, hxres: bitstring, k_seaf: key_seaf) = m3 in
  event snAuthUe(suci, SNname);
  event sn2hnOnSUCI(SNname, suci);
  event sn2hnOnKseaf(SNname, k_seaf);
  event sn2ueOnKseaf(suci, SNname, k_seaf);
  out(UE_SN, (rand, autn));
  in(UE_SN, res_star: xres_star);
  if hxres = sha256((rand, res_star)) then
    out(SN_HN, (res_star, suci));
    in(SN_HN, supi_sn: SUPI);
    event snEndWithUe(supi_sn, SNname);
    event snEndWithHnOnSUPI(SNname, supi_sn);
    event snEndWithUeOnKseaf(supi_sn, SNname, k_seaf);
    event snEndWithHnOnKseaf(SNname, k_seaf).
```

The SN process mainly preserves the scheme ordering and records representative events. These events are the anchor points of the later correspondence queries, so they are crucial for showing that a completed run on one side is matched by a genuine run on the other side, rather than by an adversarially fabricated transcript.

2.3 Verification Results and Interpretation

Representative query outcomes are summarized by the result figure reproduced below. The figure is shown only as a compact representative output; the accompanying interpretation is provided in ordinary text so that the reader can see more clearly how the reported 'true' results map to the secrecy and correspondence goals discussed in the paper.

The detailed output confirms that the attacker cannot derive k_1, k_2, sk_{hn}, $SUPI$, sqn_{ue}, sqn_{hn}, or k_{seaf}. It also validates the expected event correspondences, including the injective correspondences on k_{seaf} that bind the UE, HN,

and SN views of the same authenticated session. These results jointly support both secrecy and mutual authentication under the Dolev–Yao model.

More specifically, the secrecy queries show that the symbolic attacker never gains access to the protected subscriber identity, sequence numbers, long-term key material, or the session key seed even though the wireless-side channel is public. The correspondence queries then complement this secrecy evidence by showing that the endpoint events observed by the UE, HN, and SN cannot be completed in isolation: they are linked to one another through the event structure of the scheme run.

For completeness, the representative correspondence queries checked in the detailed appendix version include:

Listing 1.5: Representative correspondence queries.

```
query event(snEndWithUe(ue,sn)) ==> event(ueAuthSn(ue,sn)).
query event(ueEndWithHn(ue,hn)) ==> event(hnAuthUe(ue,hn)).
query event(snEndWithHnOnSUPI(sn,ue)) ==> event(hn2snOnSUPI(sn,ue)).
query inj-event(ueEndWithHnOnKseaf(ue,hn,k_seaf_4))
   ==> inj-event(hn2ueOnKseaf(ue,hn,k_seaf_4)).
query inj-event(hnEndWithUeOnKseaf(ue,hn,k_seaf_4))
   ==> inj-event(ue2hnOnKseaf(ue,hn,k_seaf_4)).
query inj-event(hnEndWithSnOnKseaf(sn,k_seaf_4))
   ==> inj-event(sn2hnOnKseaf(sn,k_seaf_4)).
```

The above queries cover both ordinary and injective correspondences. Ordinary correspondences confirm that one party's completion event implies that the peer has previously executed the matching step, while injective correspondences additionally prevent one valid run from being reused to justify multiple distinct sessions. In the context of 5G authentication, this distinction is important because it strengthens the interpretation of the symbolic result from simple reachability to authentication consistency across different runs.

3 Detailed BAN Logic Verification

Following the longer appendix version of the manuscript, the BAN-logic verification is organized into four parts: notation/rules, idealized messages, assumptions/goals, and derivations. The purpose is to show step by step how the scheme establishes the intended beliefs of the HN, UE, and SN during the initialization and challenge-response phases.

3.1 Initialization Stage

The idealized message of the initialization phase is

$$UE \to HN : \langle SUPI, UE \overset{k_2}{\Leftrightarrow} HN \rangle_{k_1}. \tag{A1}$$

The BAN-logic assumptions adopted for this stage are

$$HN \mid\equiv UE \overset{k_1}{\Leftrightarrow} HN, \tag{A2}$$

$$HN \mid\equiv \#((UE \overset{k_1}{\Leftrightarrow} HN)), \tag{A3}$$

Table 1: Core notations and rules used in the BAN-logic derivation.

Notation	Meaning	Rule	Formula
$P \mid\equiv X$	P believes X	Message Meaning (MM)	$P \mid\equiv P \leftrightarrow_K Q,\ P \triangleleft \{X\}_K \Rightarrow P \mid\equiv Q \mid\sim X$
$P \triangleleft X$	P sees X	Nonce Verification (NV)	$P \mid\equiv \#(X),\ P \mid\equiv Q \mid\sim X \Rightarrow P \mid\equiv Q \mid\equiv X$
$P \mid\sim X$	P once said X	Jurisdiction (JR)	$P \mid\equiv Q \Rightarrow X,\ P \mid\equiv Q \mid\equiv X \Rightarrow P \mid\equiv X$
$P \Rightarrow X$	P has authority over X	Freshness (FR)	$P \mid\equiv \#(X) \Rightarrow P \mid\equiv \#((X,Y))$
$\#(X)$	X is fresh	Decomposition (DR)	$P \triangleleft (X,Y) \Rightarrow P \triangleleft X$
$\langle X \rangle_K$	X combined with secret K	Belief Conjunction (BC)	$P \mid\equiv X,\ P \mid\equiv Y \Rightarrow P \mid\equiv (X,Y)$
$\{X\}_K$	X encrypted by K	Hash Rule (HR)	$P \mid\equiv Q \mid\sim H(X),\ P \triangleleft X \Rightarrow P \mid\equiv Q \mid\sim X$

$$HN \mid\equiv UE \overset{k_2}{\Leftrightarrow} HN, \tag{A4}$$

$$HN \mid\equiv \#((UE \overset{k_2}{\Leftrightarrow} HN)). \tag{A5}$$

The goal is that the HN should eventually believe that the UE itself believes the recovered identity:

$$HN \mid\equiv UE \mid\equiv SUPI. \tag{A6}$$

The derivation proceeds as follows.

$$HN \triangleleft \left\langle \{SUPI, UE \overset{k_2}{\Leftrightarrow} HN\}_{k_2}, UE \overset{k_1}{\Leftrightarrow} HN \right\rangle_{k_1}, \tag{A7}$$

$$HN \mid\equiv UE \mid\sim \left(\{SUPI, UE \overset{k_2}{\Leftrightarrow} HN\}_{k_2}, UE \overset{k_1}{\Leftrightarrow} HN\right), \tag{A8}$$

$$HN \mid\equiv UE \mid\equiv \{SUPI, UE \overset{k_2}{\Leftrightarrow} HN\}_{k_2}, \tag{A9}$$

$$HN \mid\equiv UE \mid\equiv SUPI. \tag{A10}$$

Equation (A7) follows from the received initialization message. Using the message-meaning rule with (A2) yields (A8); freshness and nonce verification with (A3) then give (A9). Finally, by combining (A9) with the belief that k_2 is a valid UE–HN secret and fresh, as assumed in (A4)–(A5), the HN concludes (A10), namely that the received $SUPI$ is endorsed by the UE.

3.2 Challenge-response Stage

The idealized messages of the challenge-response phase are written as

$$HN \to UE : RAND',\ \{SQN_{HN}\}_{AK},\ \left\langle SQN_{HN}, AMF, RAND', UE \overset{k_{seaf}}{\leftrightarrow} HN \right\rangle_{k}, \tag{A11}$$

$$UE \to SN : res^*,\ hres^*, \tag{A12}$$

$$UE \to HN : \left\langle ID_{SN}, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF \right\rangle_k, \tag{A13}$$

$$HN \to SN : \left\{ SUPI, UE \overset{k_{seaf}}{\leftrightarrow} SEAF, (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF) \right\}_{K_{N12}}. \tag{A14}$$

The assumptions used in this phase are

$$UE \mid\equiv UE \overset{k}{\leftrightarrow} HN, \tag{A15}$$
$$UE \mid\equiv X \Rightarrow UE, \tag{A16}$$
$$UE \mid\equiv \#(SQN_{HN}), \tag{A17}$$
$$UE \mid\equiv \#(k_{seaf}), \tag{A18}$$
$$SN \mid\equiv RAND', \tag{A19}$$
$$SN \mid\equiv hres^*, \tag{A20}$$
$$HN \mid\equiv UE \overset{k}{\Leftrightarrow} HN, \tag{A21}$$
$$HN \mid\equiv \#(RAND'), \tag{A22}$$
$$SN \mid\equiv SN \overset{K_{N12}}{\leftrightarrow} HN, \tag{A23}$$
$$HN \mid\equiv \#((SN \overset{K_{N12}}{\leftrightarrow} HN)), \tag{A24}$$
$$SN \mid\equiv HN \Rightarrow (UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A25}$$
$$SN \mid\equiv HN \Rightarrow (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF). \tag{A26}$$

Because k_{seaf} is regenerated from fresh ECIES/ECDH material and the SN receives $RAND'$ and $HXRES^*$ through the secure SN–HN channel, the following intermediate goals are justified:

$$UE \mid\equiv HN \mid\equiv RAND', \tag{A27}$$
$$UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A28}$$
$$UE \mid\equiv HN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A29}$$
$$HN \mid\equiv UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} HN, \tag{A30}$$
$$SN \mid\equiv HN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A31}$$
$$SN \mid\equiv UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF. \tag{A32}$$

The detailed derivation is

$$UE \triangleleft RAND', \tag{A33}$$
$$UE \mid\equiv UE \overset{AK}{\leftrightarrow} HN, \tag{A34}$$
$$UE \triangleleft \{SQN_{HN}\}_{AK}, \tag{A35}$$
$$UE \mid\equiv HN \mid\equiv SQN_{HN}, \tag{A36}$$
$$UE \triangleleft \left\langle SQN_{HN}, AMF, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF \right\rangle_k, \tag{A37}$$
$$UE \mid\equiv HN \mid\sim (SQN_{HN}, AMF, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A38}$$

$$UE \mid\equiv HN \mid\equiv (SQN_{HN}, AMF, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A39}$$
$$UE \mid\equiv HN \mid\equiv RAND', \tag{A40}$$
$$UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} HN, \tag{A41}$$
$$UE \mid\equiv HN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A42}$$
$$SN \triangleleft res^*, \tag{A43}$$
$$SN \mid\equiv res^*, \tag{A44}$$
$$HN \triangleleft \left\langle ID_{SN}, RAND', UE \overset{k_{seaf}}{\leftrightarrow} HN \right\rangle_k, \tag{A45}$$
$$HN \mid\equiv UE \mid\equiv (ID_{SN}, RAND', UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A46}$$
$$HN \mid\equiv UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} HN, \tag{A47}$$
$$SN \triangleleft \left\{ SUPI, UE \overset{k_{seaf}}{\leftrightarrow} HN, (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF) \right\}_{K_{N12}}, \tag{A48}$$
$$SN \mid\equiv HN \mid\equiv (SUPI, UE \overset{k_{seaf}}{\leftrightarrow} HN, (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF)), \tag{A49}$$
$$SN \mid\equiv HN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A50}$$
$$SN \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF, \tag{A51}$$
$$SN \mid\equiv HN \mid\equiv (UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF), \tag{A52}$$
$$SN \mid\equiv UE \mid\equiv UE \overset{k_{seaf}}{\leftrightarrow} SEAF. \tag{A53}$$

Equations (A33)–(A40) show that the UE first authenticates the freshness of $RAND'$ and then accepts the HN's belief about the current k_{seaf} binding. Equations (A41)–(A47) capture the HN-side confirmation that the UE has accepted the same session-key context. Finally, Eqs. (A48)–(A53) show how the SN, using the protected K_{N12} message and the jurisdiction assumptions in (A25)–(A26), concludes both that $UE \overset{k_{seaf}}{\leftrightarrow} SEAF$ holds and that the UE itself believes this binding. Hence, the challenge-response phase establishes mutual authentication together with a shared belief in the freshness and legitimacy of k_{seaf}.

4 Detailed Provable-Security Analysis

This section gives the complete ROR-based provable-security analysis that is summarized in the main paper. The analysis focuses on session-key security and models the adversary as a probabilistic polynomial-time algorithm with control over the public communication channel.

4.1 Security Model

As indicated below, the adversary's abilities and conduct are formalized through a collection of symbolic notations and query mechanisms.

Players: The proposed scheme involves three types of entities: User Equipment (UE), Serving Node (SN), and Home Node (HN). Let

$$\Pi = \{UE^p, SN^q, HN^r \mid p, q, r \in \mathbb{N}\} \tag{2}$$

denote the set of all scheme instances. Here, UE^p is the p-th session instance of the user equipment, responsible for initiating authentication and key negotiation; SN^q is the q-th session instance of the serving node, acting as an intermediate relay for authentication messages; and HN^r is the r-th session instance of the home node, responsible for generating authentication vectors and verifying user legitimacy.

Queries: The adversary $\mathcal{A}$ is modeled as a probabilistic polynomial-time (PPT) algorithm with full control over the public communication channel. The adversary's capabilities are formalized through the following query types:

- Execute(UE^p, HN^r): The adversary observes and records all messages transmitted between the matching instances UE^p and HN^r during the scheme execution. The output is the message sequence $\{M_1, M_2, \ldots, M_n\}$ exchanged in the session.
- Reveal(M^t): If instance M^t has established a session key k_{seaf}, where $M \in \{UE, SN, HN\}$, the query returns k_{seaf}; otherwise, it returns $\perp$.
- Corrupt(M^t): The query returns the long-term secret of instance M^t, including the UE private key SK_{UE} or the HN private key SK_{UDM}. Temporary secrets of established sessions are not revealed.
- Test(M^t): This query may be issued at most once and only to a fresh session. The challenger returns the real session key k_{seaf} if $b = 1$, and otherwise returns a random string of the same length, where $b \in \{0, 1\}$.

Freshness: An instance M^t is fresh if neither it nor its matching session has been exposed through *Corrupt*, and if the exchanged messages have not been tampered with or replayed by the adversary.

4.2 Security Proof

The formal security theorem and its proof are given below.

Theorem. Let $\mathcal{A}$ be a PPT adversary attempting to break the session key security of the proposed scheme. Let q_h be the number of hash queries executed by $\mathcal{A}$, let l be the output length of the one-way hash function $H(\cdot)$, and let $Adv_{\mathcal{A}}^{ECDLP}$ denote the advantage of $\mathcal{A}$ in solving the elliptic-curve discrete logarithm problem. Then the security advantage of $\mathcal{A}$ satisfies

$$Adv_{\mathcal{A}} \leq 2Adv_{\mathcal{A}}^{ECDLP} + \frac{q_h^2}{2^l}. \tag{3}$$

Since both $Adv_{\mathcal{A}}^{ECDLP}$ and $\frac{q_h^2}{2^l}$ are negligible for PPT adversaries, the proposed scheme is provably secure under the ROR model.

Proof: Consider games G_0–G_3. G_0 is the real attack game; G_1 replaces the hash function with a random oracle; G_2 replaces the ECDH-derived secret with a random value; and G_3 replaces the session key with a uniform random string. The transition from G_1 to G_2 is bounded by $Adv_{\mathcal{A}}^{ECDLP}$, and that from G_2 to G_3 by $\frac{q_h^2}{2^{l+1}}$. Since $\Pr[Succ_{G_3}] = \frac{1}{2}$, we obtain the following bound, which proves security under the ROR model:

$$Adv_{\mathcal{A}} \leq 2Adv_{\mathcal{A}}^{ECDLP} + \frac{q_h^2}{2^l}.$$

5 Closing Remark

This supplementary document retains the detailed heuristic analysis, ProVerif model, BAN-logic materials, and ROR-based provable-security analysis omitted from the conference version. Read together with the main paper, it provides a complete presentation of the scheme's security reasoning, symbolic-verification evidence, belief-level analysis, and game-based security proof.